Kandidatnr: 60

Eksamen – Skysikkerhet

Desember 2024

Task A

Plan:

Steg 1: Migrere applikasjon til sky

- Bruke google cloud som provider
- Opprette et nytt prosjekt i Google Cloud SDK Shell.
- Angi prosjekt-ID (sky2100eksamen)

Steg 2: Sette opp en virtuell maskin (VM):)

- Opprette en VM med navnet strelizia-vm
- Bruke maskintype e2-micro for å minimere kostnader, ettersom Strelizia er svært begrenset på penger.
- Bruke Debian-12 som operativsystem da dette gir stabilitet og sikkerhet.
- Legge til tags som kan brukes i brannmurregel.
- Bruke standard nettverket i Google Cloud for å opprette en ekstern IP-adresse som gjør VM-en tilgjengelig over internett.

Steg 3: Konfigurere http-tilgang

- Opprette en brannmurregel som tillater http-trafikk.
- Tillate protokoll: tcp og port: 80 for standard http trafikk.
- Tillate trafikk fra alle IP-adresser ved å bruke secure_ranges.
- Bruke target_tags «http» for å være sikker på at regelen kun gjelder den VM-en som trenger http-tilgang.

Steg 4: Konfigurere en Web Application Firewall

- Opprett en sikkerhetspolicy som beskytter applikasjonen mot vanlige trusler som Cross-Site Scripting og SQL-injection.
- Lage regler som blokkerer forespørsler med skadelig script eller SQL-spørringer.
- Opprett en backend-tjeneste som kobler sikkerhetspolicyen til applikasjonen, som vil fungere som en kobling mellom WAF og applikasjonen som kjører på VM-en.
- Opprett en health checker som overvåker backend-tjenesten.

Steg 5: Opprette en backup

- Implementer en backup-løsning ved å ta snapshots av boot-disken til den virtuelle maskinen.

Steg 6: Dokumentere hele prosessen:

- Utføre dokumentasjon av hele prosessen, inkludert planlegging, konfigurasjon og validering.

Utførelse:

C: > cloudsikkerhet > SKY2100 Eksamen > main.tf > ...

```
# Provider - google cloud
provider "google" {
    project = "sky2100eksamen"
    region  = "europe-north1"
}

# VM
resource "google_compute_instance" "strelizia-vm" {
  name         = "strelizia-vm"
  machine_type = "e2-micro"
  zone         = "europe-north1-a"

  allow_stopping_for_update = true # Dette tillater terraform å stoppe VM-en for å oppdatere

  tags = ["http","strelizia"]

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12" # operativsystem for VM-en
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  shielded_instance_config {
  enable_secure_boot          = true # Hindrer oppstart fra uautoriserte systemer
  enable_vtpm                 = true # aktiverer vtpm
  enable_integrity_monitoring = true # Overvåker for kompromittering
  }
}
```

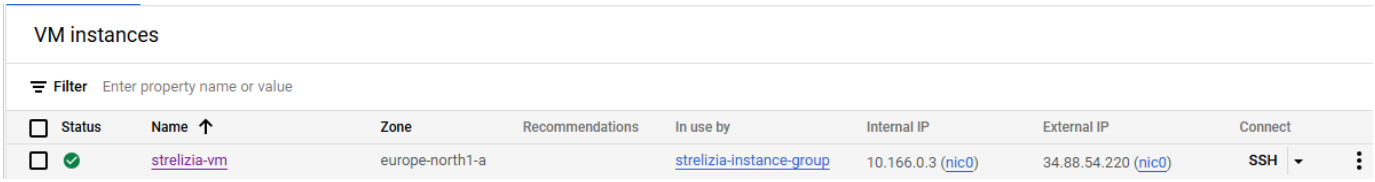
Jeg fulgte i stor grad den opprinnelige planen min, men gjorde noen justeringer underveis for å forbedre sikkerheten og gjøre koden med oversiktlig.

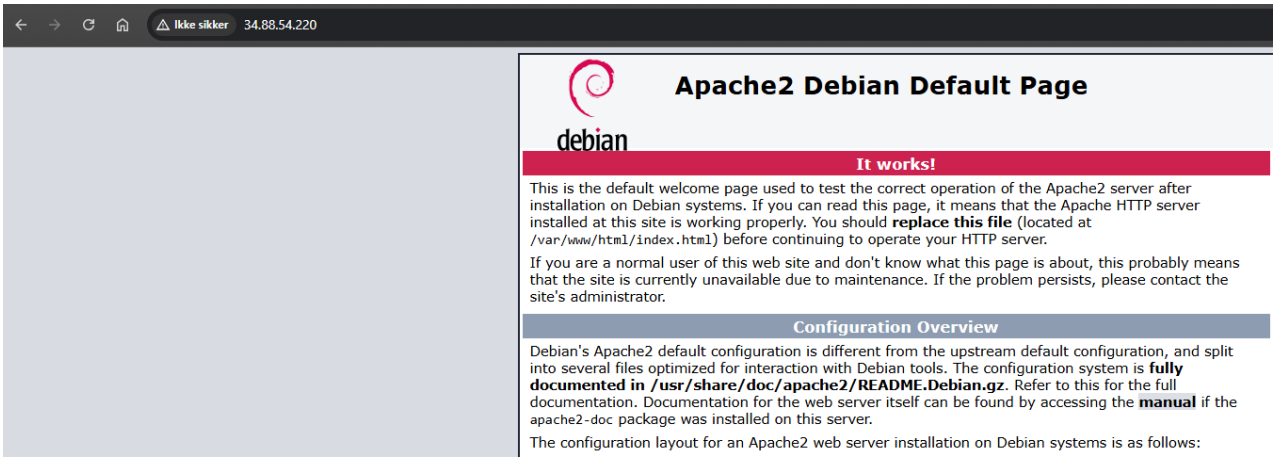
I min opprinnelige plan nevnte jeg ikke bruk av shielded_instance_config, men etter å ha lest gjennom oppgaven en gang til valgte jeg å inkludere dette for å øke beskyttelsen av VM-en. Dette ble gjort på grunn av selskapets behov for sikkerhet og beskyttelse av kritiske systemer. Jeg oppdaget også underveis at det var nødvendig å bruke allow_stopping_for_update for å kunne oppdatere VM-en underveis. Dette gjør det mulig å foreta endringer uten å måtte stoppe og starte VM-en, noe som bidrar til en mer automatisert infrastruktur.

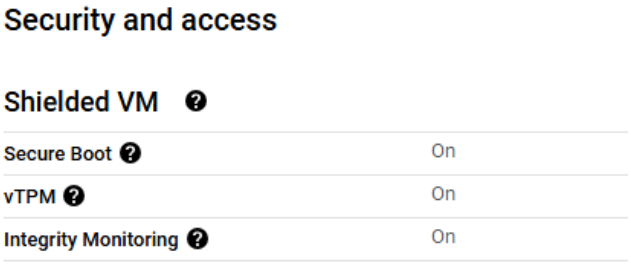
Jeg planla opprinnelig å ha hele koden samlet i en main.tf fil, men dette viste seg til å bli rotete og uoversiktlig etter hvert som prosjektet ble større. Jeg valgte derfor å dele opp prosjektet i flere .tf filer. Dette gjorde prosjektet mye mer oversiktlig og enklere å lese.

Kandidatnr: 60

Dokumentasjon:







HTTP-trafikk:

```
# HTTP-trafikk
resource "google_compute_firewall" "allow-http" {
    name    = "allow-http"
    network = "default"

  allow {
    protocol = "tcp"
    ports    = ["80"] # Tillater http og egendefinert port
  }

  source_ranges = ["0.0.0.0/0"] # Tillater trafikk fra alle IP adresser
  target_tags   = ["http"]      # Knytter regelen til VM med "http" taggen

}
```

Opprettelsen av brannmurregelen allow-http for å tillate http-trafikk gikk relativt greit uten noen komplikasjoner. Denne reglen tillater trafikk på port 80, slik at strelizia-vm kan kommunisere via http.

Dokumentasjon:

Filter Enter property name or value

Name	Type	Targets	Filters	Protocols / ports	Action	Priority	Network
allow-http	Ingress	http	IP ranges: 0.0.0.0/0	tcp:80	Allow	1000	default

```
C:\Users\[redacted]>nmap -p 80 34.88.54.220
Starting Nmap 7.95 ( https://nmap.org ) at 2024-12-05 20:56 Vest-Europa (normaltid)
Nmap scan report for 220.54.88.34.bc.googleusercontent.com (34.88.54.220)
Host is up (0.025s latency).

PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 0.44 seconds
```

Det ble utført en nmap-skanning på IP-adressen 34.88.54.220 for å bekrefte at port 80 er åpen/tilgjengelig.

Web Application Firewall (WAF):



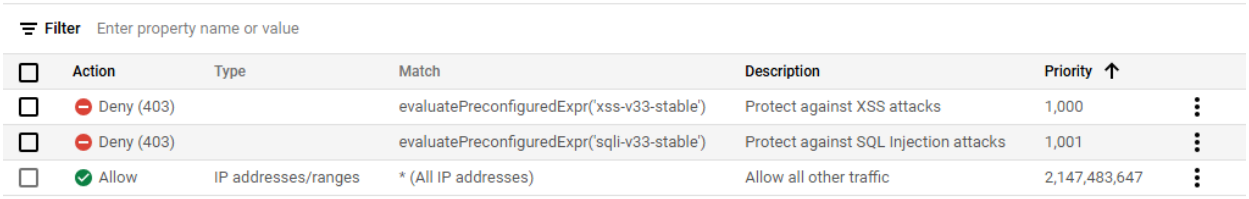
I oppgaven ble det forklart «you need to find a solution on how to protect the legacy applications from the most common attacks». Basert på min vurdering er Cross-Site Scripting og SQL injection de to mest vanlige angrepene en webapplikasjon er utsatt for. For å beskytte mot disse truslene, implementerte jeg en WAF som automatisk blokkerer disse type angrep.

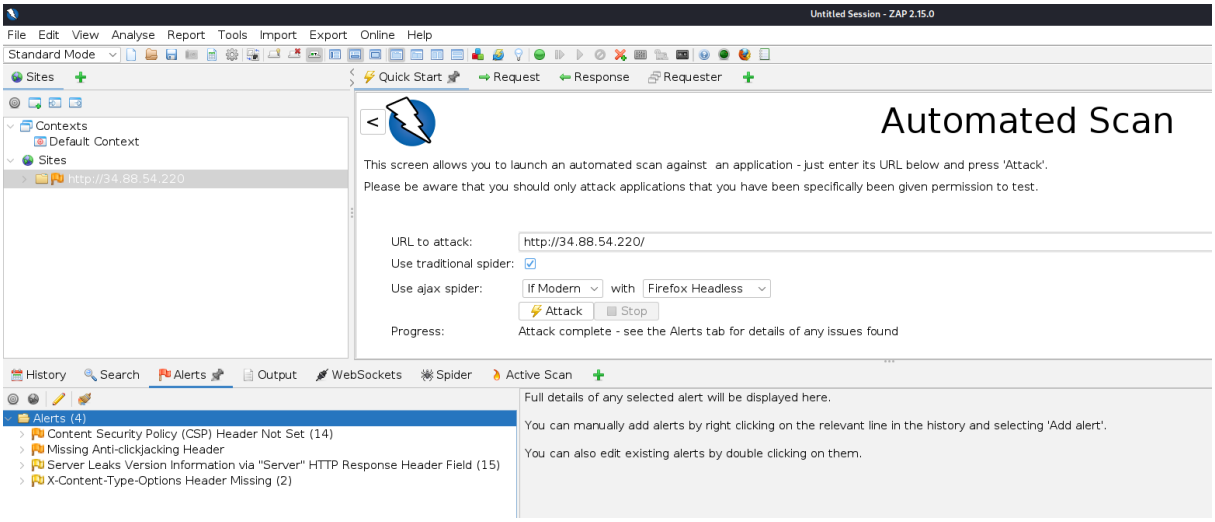
I tillegg opprettet jeg en standard regel som tillater all annen trafikk. For at denne løsningen skal fungere, er det også nødvendig med en backend-tjeneste. Dette gjør at WAF policyen kan kobles til backend tjenesten, noe som sikrer at all trafikk som går gjennom WAF blir filtrert før det når applikasjonen.

Kandidatnr: 60

Dokumentasjon:

Skjermbildet under viser WAF-regelen som ble satt opp i Google Cloud for å beskytte mot XSS og SQL injection, i tillegg viser det en standardregel som tillater all annen trafikk.





Ingen sårbarheter relatert til Cross-Site Scripting eller SQL-injection ble oppdaget under OWASP ZAP skanning. Dette indikerer at de implementerte WAF-reglene fungerer og beskytter applikasjonen mot XSS og SQL-injection.

Kandidatnr: 60

Backend:

main.tf | waf.tf | backend.tf | health.tf | backup.tf

C: > cloudsikkerhet > SKY2100 Eksamen > backend.tf > ...

```hcl
# Backend Service
resource "google_compute_backend_service" "strelizia_backend" {
  name            = "strelizia-backend"
  protocol        = "HTTP"
  timeout_sec     = 10
  health_checks   = [google_compute_http_health_check.default.id]

  backend {
    group = google_compute_instance_group.default.self_link
  }  # Kobler backend tjenesten til en instance group

  # Kobler WAF policyen til backend tjenesten
  security_policy = google_compute_security_policy.xxs_sql_protecion.id
}

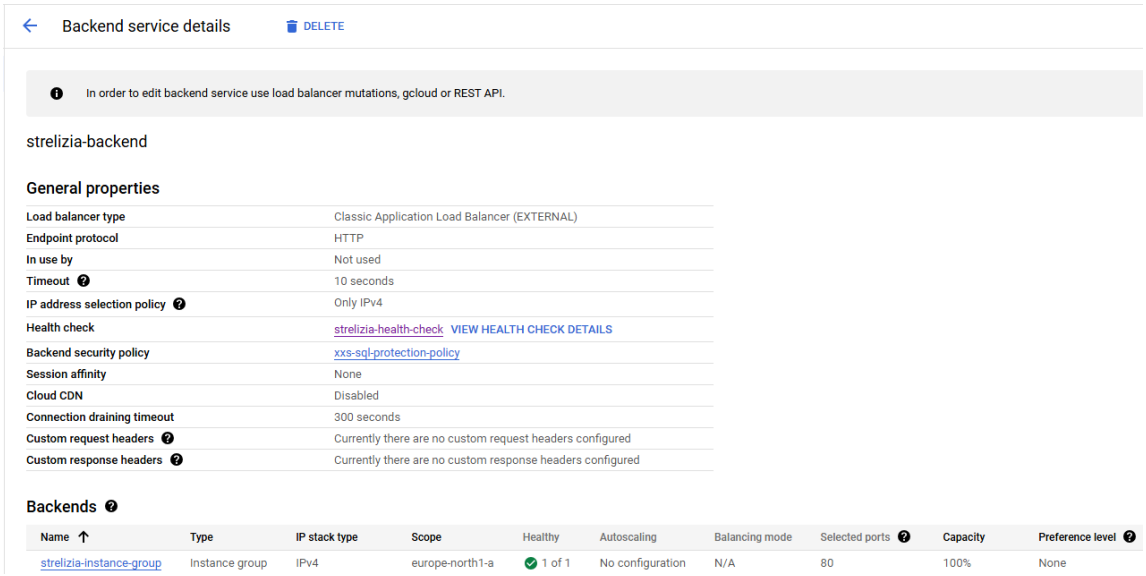
# Instance group
resource "google_compute_instance_group" "default" {
  name = "strelizia-instance-group"
  zone = "europe-north1-a"

  instances = [
    google_compute_instance.strelizia-vm.self_link # Legger til VM i instance group
  ]

  named_port {
    name = "http"
    port = 80
  }
}
```

Koden her viser hvordan backend-tjenesten strelizia-backend er satt opp. Denne er ansvarlig for å rute trafikk til strelizia-vm. Her ble mye av den opprinnelige planen brukt, men jeg gjorde noen justeringer. For å håndtere trafikken effektiv opprettet jeg en instance group, slik at backend-tjenesten kan koble seg til strelizia-vm, som trafikken kan gå gjennom.

Skjermbildet under viser backend-tjenesten i Google Cloud Console:



Kandidatnr: 60

Health check:

main.tf | waf.tf | backend.tf | health.tf | backup.tf

C: > cloudsikkerhet > SKY2100 Eksamen > health.tf > ...

```
1   # Health Check
2   resource "google_compute_http_health_check" "default" {
3     name                = "strelizia-health-check"
4     request_path        = "/"
5     check_interval_sec  = 3
6     timeout_sec         = 3
7     healthy_threshold   = 2
8     unhealthy_threshold = 2
9
10  }
```

Dette viser hvordan Health-check er konfigurert for å overvåke backend-tjenesten.

Dokumentasjon:

For å verifisere om health check fungerer som det skal, stoppet jeg apache-tjenesten ved å skriv inn sudo systemctl stop apache2. Da tjenesten stoppet, ble de vist "0 of 1" (unhealthy) på backend statusen. Det vil si at health check indentifiserte at tjenesten ikke var tilgjengelig. Deretter startet jeg apache-tjenesten igjen, da viste backend statusen, "1 of 1" (healthy). Dette bekrefter at alt fungere som det skal.

SSH-in-browser UPLOAD FILE DOWNLOAD FILE

```
Linux strelizia-vm 6.1.0-27-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.115-1 (2024-11-01) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Dec  5 15:52:54 2024 from 35.235.244.178
[redacted]@strelizia-vm:~$ sudo systemctl stop apache2
[redacted]@strelizia-vm:~$ sudo systemctl start apache2
[redacted]@strelizia-vm:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Thu 2024-12-05 23:55:42 UTC; 35s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 3008 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 3012 (apache2)
      Tasks: 55 (limit: 1136)
     Memory: 13.3M
        CPU: 97ms
     CGroup: /system.slice/apache2.service
             ├─3012 /usr/sbin/apache2 -k start
             ├─3013 /usr/sbin/apache2 -k start
             └─3014 /usr/sbin/apache2 -k start

Dec 05 23:55:42 strelizia-vm systemd[1]: Starting apache2.service - The Apache HTTP Server...
Dec 05 23:55:42 strelizia-vm systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Backends

Name	Type	IP stack type	Scope	Healthy	Autoscaling	Balancing mode	Selected ports	Capacity	Preference level
strelizia-instance-group	Instance group	IPv4	europe-north1-a	⚠ 0 of 1	No configuration	N/A	80	100%	None

Backends

Name	Type	IP stack type	Scope	Healthy	Autoscaling	Balancing mode	Selected ports	Capacity	Preference level
strelizia-instance-group	Instance group	IPv4	europe-north1-a	✓ 1 of 1	No configuration	N/A	80	100%	None

Kandidatnr: 60

Backup:

main.tf | waf.tf | backend.tf | health.tf | backup.tf

C: > cloudsikkerhet > SKY2100 Eksamen > backup.tf > ...

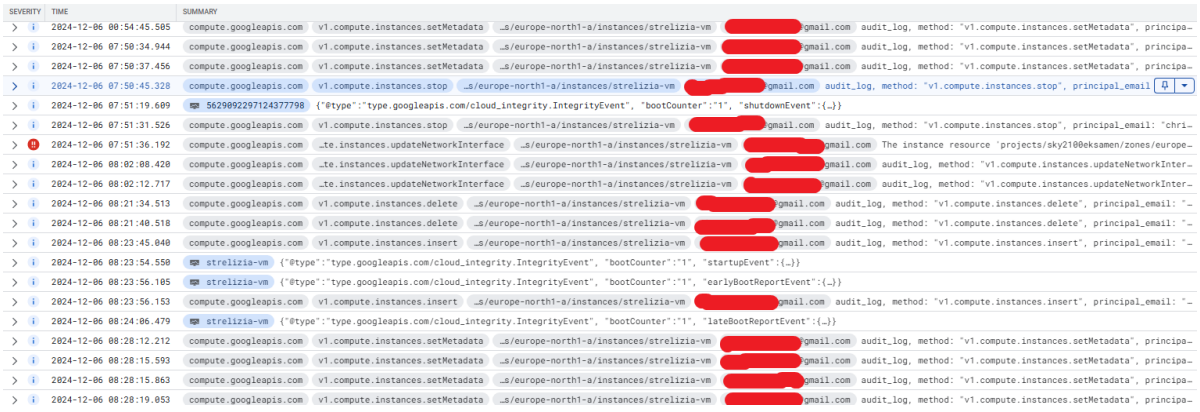
```
1  # Backup-løsning
2
3  resource "google_compute_snapshot" "strelizia_vm_snapshot" {
4    name = "strelizia-vm-snapshot"
5    source_disk = google_compute_instance.strelizia-vm.boot_disk[0].source
6    zone = "europe-north1-a"
7  }
```

Dette viser hvordan jeg opprettet en backup-løsning for strelizia-vm ved hjelp av google cloud snapshots. Dette gir en sikkerhetskopi av boot disken til strelizia-vm, som kan brukes gjenopprette systemet ved tap av data.

Filter Enter property name or value

Status	Name	Location	Snapshot size	Creation time	Creation type	Source disk	Source instant snapshot	Disk size	Architecture
✓	strelizia-vm-snapshot	eu	645.54 MB	Dec 5, 2024, 4:43:58 PM UTC+01:00	Manual	strelizia-vm		10 GB	–

Task A. 2



Her kan du se loggene som genereres ved distribusjon og drift av infrastrukturen. Loggene viser ulike handlinger som stop og start av instanser, instance.delete og instances.insert, som er opprettelse og sletting av VM. I tillegg kan vi se oppdatering av nettverksgrensesnitt.

For å få en enda bedre overvåkning og analyse av disse loggene kan det integreres med SIEM-verktøy som f.eks. splunk eller IBM QRadar. Ved å bruk API-er, kan man transportere loggene fra google cloud til SIEM-verktøyene. Splunk og IBM vil da identifiser forskjellige mønstre i loggdataen. De kan da varsle om uvanlige hendelser, som bidrar til en rask respons.

Ved å integrere loggene med et SIEM-verktøy vil man få flere fordeler, man vil kunne identifisere og respondere på trusler i sanntid. SIEM skanner loggene konstant og legger merker til mistenkelig aktivitet med engang, noe som gir muligheten til å reagere raskt og forhindre skade. Dersom en VM skulle stoppes på en uventet måte, vil SIEM umiddelbart varsle om dette. I tillegg gir SIEM muligheten til å overvåke brukere, apper og enheter, noe som øker sikkerheten på hele infrastrukturen.

Task B

How does cloud services work? What is the biggest benefit of using cloud services from a financial perspective?

En skytjeneste er en nettbasert tjeneste som gir deg mulighet til å lagre data via internett. Dataen lagres ikke lokalt, men på servere som administreres av en skytjenesteleverandør. Eksempel på skytjenester er oneDrive, google Drive og icloud.

Når du lagrer noe i skyen, blir dataen sendt til et stort datasenter som består av flere servere. Her blir dataen lagret på en sikker måte, ofte med flere kopier. Skytjenester bruker eksterne servere til å lagre data som bilder, filer og videoer. Når data lastes opp til skyen, blir den lagret på en virtuell maskin som kjører på en fysisk server.

Det er flere fordeler med å bruke skytjenester, men fra et økonomisk perspektiv er den største fordelen at det reduserer kostnader og øker effektiviteten. Selskaper slipper å gjøre store investeringer i servere og utstyr, samtidig som de får tilgang til nesten ubegrenset kapasitet og lagring. Ved å kun betale for den kapasiteten som faktisk brukes, ungår bedrifter unødvendige utgifter, noe som gjør skytjenester til en meget kostnadseffektiv løsning.

https://cloud.google.com/learn/what-is-cloud-storage?hl=en

https://www.advania.no/kunnskap/artikler/fordeler-og-ulemper-med-skytjenester

What is the benefits from a technical perspective of using cloud services?

Fra et teknisk perspektiv gir skytjenester flere fordeler:

Skalerbarhet og fleksibilitet, man kan enkelt tilpasse ressursene som lagring og prosessorkraft, for å møte bedriftens behov. Dette gjør det mulig å håndtere økende arbeidsbelastning effektiv, samtidig som ressurser blir tilpasset etter bedriftens behov uten noe unødvendig overforbruk.

Forebygging av datatap, skytjenester tilbyr viktige funksjoner som sikkerhetskopiering og datagjenoppretting. Dette redusere risikoen for datatap ved cyberangrep, systemfeil eller brukerfeil.

Avansert sikkerhet, skytjenester bruker IAM (Identity and Access Management) for å sikre at kun autoriserte brukere og systemer har tilgang. Dette blir ofte kombinert med MFA, som legger til flere nivåer av sikkerhet ved å kreve identitetsbekreftelse som passord og engangskode. I tillegg krypteres all data som sendes, mottas og lagres skyen.

https://cloud.google.com/learn/advantages-of-cloud-computing?hl=en

What is the difference between the different operating models within a cloud service? (IaaS, PaaS, SaaS). Who has responsibility of what?

IaaS (Infrastructure as a service) er levering av datainfrastruktur som lagring, virtualisering og nettverkstjenester over internett. Kunden trenger ikke å administrere eller vedlikeholde maskinvare eller virtualisering da dette blir tatt hånd om av skyleverandøren. Kunden har ansvar for å konfigurere og vedlikeholde operativsystem, applikasjoner, skalering og runtime (Google Cloud, 2024). Eksempel på IaaS leverandører er: Microsoft Azure, AWS og google Compute Engine.

PaaS (Plat as a service) er en tjeneste som leverer og administrerer maskinvare og programvare, og gir utviklere et miljø til å utvikle og teste applikasjoner via cloud. Her har kunden ansvar for applikasjonene, data og skalering, mens skytjenesten har ansvar for operativsystem, runtime, virtualisering og maskinvare (Google Cloud, 2024). Eksempler på PaaS-leverandører er Google App Engine, Windows Azure og force.com.

SaaS (Software as a Service) er en tjeneste der brukere får tilgang til ferdig utviklet applikasjoner. SaaS administreres fullstendig av tjenesteleverandøren (Google Cloud, 2024). Fordi SaaS-applikasjoner kan åpnes direkte gjennom en nettleser, er det ikke nødvendig for kunden å laste ned eller installere noe på enhetene sine. Eksempel på Saas-tjenester er Google Workspace, Microsoft Office 365 og Salesforce.

https://www.datatilsynet.no/personvern-pa-ulike-omrader/internett-og-apper/skytjenester/

https://cloud.google.com/learn/paas-vs-iaas-vs-saas?hl=en

How are cloud services secured? Is there any public examples of where cloud services were not secured properly?

En av mange grunner til at skytjenester er sikre, er bruken av IAM (Identity and Access Management). IAM sørger for at kun autoriserte brukere og systemer har tilgang til skyen. Dette blir ofte kombinert med MFA (multifactor-autorisering) for å styrke sikkerheten enda mer, ved å kreve flere lag med identitetsbekreftelse, som passord og engangskode. I tillegg krypteres all data som sendes, mottas og lagres i skyen ved hjelp av krypteringsalgoritmer, ofte AES-256 som er en standard. Dette sikrer at dataen forblir utilgjengelig selv om den skulle bli fanget opp under transitt. All data blir også automatisk sikkerhetskopiert, slik at den kan gjenopprettes.

Et kjent eksempel på at en skytjeneste ikke var sikret riktig er Facebook lekkasjen I 2019, der over 540 millioner brukere ble eksponert. Dette skjedde på Amazons sin AWS og skyldtes at tredjepartsutviklere lagret data i feilkonfigurerte «S3 bucket». Opplysninger som ble lekket under denne lekkasjen var bruker-ID, navn og annen sensitiv informasjon.

https://www.bleepingcomputer.com/news/security/540-million-facebook-records-leaked-by-public-amazon-s3-buckets/

What is a Cloud Native application?

En cloud native applikasjon er en spesialdesignet programvare, som er utviklet for å utnytte fordelene med skytjenester, som fleksibilitet og scaling. I motsetning til tradisjonelle monolittiske applikasjoner, bestående av en stor enhet, er cloud native applikasjoner delt opp i mindre, mikrotjenester. Disse tjenestene fungerer ved hjelp av moderne teknologi og metoder som DevOps, kontinuerlig integrasjon og levering, containere og deklarative APIs (Google Cloud, 2024). Dette gjør det mulig å distribuere og skalere komponenter, noe som gir fleksibilitet til å gjennomføre oppdateringer og løse problemer uten å påvirke tjenestens tilgjengelighet.

https://cloud.google.com/learn/what-is-cloud-native?hl=en

Define and explain the different cloud migration strategies

Rehost (Lift and Shift) er en strategi som handler om å gjøre så få endringer som mulig under en migrering. Dette er den enkleste skymigreringsstrategien, der applikasjoner, virtuelle maskiner og serveroperativsystemer flyttes direkte fra lokal infrastruktur til en offentlig skyplattform uten endringer (Google Cloud, 2024). Målet her er å flytte applikasjonene så raskt og enkelt som mulig, uten å påvirke koden eller strukturen. Ulempen med denne strategien er at den ikke får utnyttet skyens fulle potensial.

Replatforming (Lift and Optimize) er en strategi som handler om å flytte applikasjoner og virtuelle maskiner til skyen, for deretter å optimalisere dem for det nye miljøet. Målet er å oppnå bedre ytelse og effektivitet uten å gjøre store endringer i applikasjonens struktur. Denne strategien krever mer tid og ressurser enn en enkel «Lift and Shift».

Repurchasing, denne strategien handler om å erstatte eksisterende applikasjoner med skybaserte løsninger som f.eks. SaaS (Google Cloud, 2024). Dette innebærer å bytte lokale systemer med moderne skytjenester som tilbyr bedre funksjonalitet og brukervennlighet. Denne strategien reduserer behovet for vedlikehold, og gjør det enklere å omstrukturere, i tillegg til at den krever færre ressurser på lengre sikt.

Refactoring (move and improve), dette er en strategi for å rekonstruere eksisterende applikasjoner til skybaserte løsninger. Dette innebærer å endre koden uten å påvirke applikasjonens frontend eller brukeropplevelse, med mål om å dra nytte av skybaserte funksjoner som fleksibilitet, skalerbarhet og forbedret ytelse. Denne strategien er svært kompleks og ressurskrevende, men gir muligheten for optimalisering og bedre ytelse.

Retaining innebærer å beholde applikasjoner på den eksisterende infrastrukturen i stedet for å flytte dem til skyen, på grunn av sikkerhetsmessige eller tekniske årsaker. Målet med retaining er å sikre stabilitet og kontinuitet for kritiske systemer, mens andre deler av infrastrukturen oppdateres.

Retiring, denne strategien handler om å bruke migreringsprosessen som en anledning til å identifisere ubrukte eller utdaterte applikasjoner. Dette reduserer kostnader og kompleksitet ved å fjerne unødvendige applikasjoner.

https://cloud.google.com/learn/cloud-migration?hl=en

What is a Web Application Firewall? What would one use such a firewall for?

En Web Application Firewall (WAF) er en slags sikkerhetsmekanisme som beskytter webapplikasjoner ved å blokkere skadelig forespørsler. Den brukes til å beskytte webapplikasjoner mot trusler som Cross-Site Scripting (XSS) og SQL-injection. Den analyserer og filtrerer trafikken mellom brukere og webapplikasjonen, og stopper mistenkelige forespørsler. Et eksempel på dette kan være dersom en angriper prøver å få tilgang til sensitiv informasjon ved å utføre XSS eller SQL-injection. WAF vil oppdage dette og blokkere forespørselen før den når webapplikasjonen.

https://www.f5.com/glossary/web-application-firewall-waf

Define and explain what DevSecOps is, and how this differs compared to more traditional product development approaches.

DevSecOps står for Development, Security, og Operations, og innebærer å integrere sikkerhet i alle prosesser innen programvareutvikling. Målet med DevSecOps er å fordele ansvaret for sikkerhet mellom utviklere, sikkerhetsteam og driftsteam, slik at sikkerhet blir en naturlig del av hele utviklingsprosessen. DevSecOps oppnås gjennom et tett samarbeid mellom de forskjellige teamene, kombinert med tydelige arbeidsprosesser og automatisering av sikkerhetsoppgaver. Ved å dele ansvaret for sikkerhet mellom alle, blir sikkerhet sett på som en essensiell praksis i stedet for et enkelt steg man gjør på slutten.

Sammenlignet med mer tradisjonelle utviklingsmetoder integrerer DevSecOps sikkerhet i hele utviklingsprosessen, i stedet for at det behandles som en oppgave man gjør helt på slutten. Tradisjonelle metoder fokuserer ofte på å adressere sikkerhet etter at koden er skrevet og testet, noe som kan føre til forsinkelser og økte kostnader dersom det bliroppdaget feil sent i prosessen. DevSecOps løser dette fint ved å integrere sikkerhet helt fra starten av, noe som reduserer risikoen og øker effektiviteten gjennom kontinuerlig testing.

https://www.microsoft.com/nb-no/security/business/security-101/what-is-devsecops

https://aws.amazon.com/what-is/devsecops/

Using Azure, AWS or GCP. What is the most essential knowledge one should acquire to use these cloud providers in a secure way? Is there any best-practices, or architecture guides?

Dersom man skal bruke Azure, AWS eller GCP, er det særdeles viktig å ha en god forståelse av grunnleggende skysikkerhet. De viktigste kunnskapene man bør ha for å bruke skytjenester på en sikker måte er IAM, logging og nettverksikkerhet.

Bruken av IAM (Identity and Access Management) er kanskje en av de viktigste faktorene for å sikre et godt skymiljø. Dette gir kontroll over hvem som har tilgang systemet.

Det er også viktig at man implementerer god logging for å kunne undersøke mistenkelig hendelser. Bruk av et SIEM-verktøy vil gjøre det mye enklere å håndtere sikkerhetsbrudd.

En sikker konfigurasjon av nettverk er også viktig. Ta i bruk brannmurer og VPN for å beskytte data.

Jeg ville anbefalt å bruke google sin Cloud Architecture Center, https://cloud.google.com/architecture/security-foundations.

What are the downsides of using Cloud? Is there any benefit of not using cloud?

Det finnes også ulemper med å bruke skytjenester. En av de største utfordringene er avhengigheten av internett. Skytjenester krever en stabil internettforbindelse, og hvis denne svikter, mister man tilgang til filer og tjenester. Dette gjør det avgjørende å ha en pålitelig internettforbindelse for å unngå problemer med tilgjengelighet.

Selv om skytjenester generelt er svært sikre, er de likevel utsatt for dataangrep som kan føre til lekkasje av sensitiv informasjon. I tillegg kan de være sårbare for DDoS-angrep, hvor noen overbelaster tjenesten, noe som kan hindre brukere i å få tilgang til ressursene sine.

En fordel med å ikke bruke skytjenester er at man har alt lokalt og dermed full kontroll over dataen.

https://www.opit.com/magazine/advantages-of-cloud-computing/